Recognizing Dynamic Programming: A General Method

Programming and Data Structures — Week 9, Lecture 4

Dr. Innocent Nyalala

Objectives

By the end of this lecture, you will be able to state a general five-step method for recognizing whether a new problem admits a dynamic programming solution and for constructing that solution systematically; apply that method to a genuinely new problem, coin change, working through it from scratch; explain precisely why naive greedy reasoning fails on coin change in general, directly previewing the central distinction Week 10 draws between greedy and dynamic programming; and consolidate every dynamic programming problem covered this week into a single comparison table.

Outline

Today covers a locksmith's-checklist analogy for systematic problem recognition; the general five-step method for identifying and solving a dynamic programming problem; a live application of that method to the coin change problem, worked from scratch; a concrete counterexample showing why greedy reasoning fails on coin change in general; a consolidated comparison table of every DP problem covered this week; the MTech-track preview of dynamic programming on tree structures; an in-class quiz; and a summary.

The locksmith's-checklist analogy

A locksmith encountering an unfamiliar lock does not improvise randomly — they run a systematic checklist, identifying the lock type, the applicable tools, and what has worked on similar locks encountered before. This lecture provides exactly that kind of checklist for recognizing and solving dynamic programming problems: a repeatable procedure, not an unteachable flash of insight available only to some students. Applying it correctly and consistently is a learnable skill, exactly parallel to the structure-choice pattern-recognition Week 8 Lecture 4 built for choosing between data structures.

The five-step method, stated generally

The general method has five steps. First, define the subproblem precisely, stating in plain words exactly what f of some parameters means — this step alone resolves most confusion, and rushing past it is the most common source of an incorrect recurrence. Second, find the recurrence relating f to strictly smaller instances of itself. Third, identify the base case or cases — the smallest subproblems, answerable directly with no further recursion. Fourth, decide the fill order for tabulation, or add a cache for memoization, ensuring every dependency a computation needs is already available when it is needed. Fifth, reconstruct the actual solution if required, not merely the optimal value, using the backward-walk technique demonstrated in Lecture 2 and Lecture 3.

Applying it live: the coin change problem

Applying the five-step method live to a genuinely new problem: given coin denominations $[1, 3, 4]$ and a target amount, find the minimum number of coins summing to exactly that amount. Step 1 defines the subproblem precisely: $f(a)$ is the minimum number of coins needed to make amount a exactly. Step 2 derives the recurrence: for every coin denomination c that fits within a, one strategy is using that coin once and then optimally solving the remaining amount $a - c$; trying every available coin and taking the option requiring the fewest total coins gives $f(a) = 1 + \min_{c \leq a} f(a - c)$.

Steps 3-5, completing the method

Step 3 identifies the base case: $f(0) = 0$, since zero coins are needed to make an amount of zero. Step 4 determines the fill order: computing $f(1), f(2), f(3)$, and onward in increasing order guarantees every $f(a - c)$ term needed by the recurrence is already computed, since $c \geq 1$ always makes $a - c$ strictly smaller than a. Step 5 handles reconstruction: recording which specific coin achieved the minimum at each amount, then walking backward from the target amount, subtracting the recorded coin at each step, exactly the same backward-walk principle used in Lecture 2 and Lecture 3.

Building the table, completely, for target amount 6

```
f(0) = 0
f(1) = 1 + min(f(0)) = 1 + 0 = 1          [only coin 1 fits]
f(2) = 1 + min(f(1)) = 1 + 1 = 2          [only coin 1 fits]
f(3) = 1 + min(f(2), f(0)) = 1 + min(2,0) = 1     [coin 1 → f(2)=2, coin 3 → f(0)=0]
f(4) = 1 + min(f(3), f(1), f(0)) = 1 + min(1,1,0) = 1  [coin 4 → f(0)=0]
f(5) = 1 + min(f(4), f(2), f(1)) = 1 + min(1,2,1) = 2  [coin 1→f(4)=1, or coin 4→f(1)=1]
f(6) = 1 + min(f(5), f(3), f(2)) = 1 + min(2,1,2) = 2  [coin 3 → f(3)=1]
```

Building the complete table up to target amount 6: notice $f(3) = 1$ directly, since the single coin of denomination 3 suffices, cheaper than using three coin-1s. $f(4) = 1$ directly, using the single coin of denomination 4. $f(6) = 2$, achieved by using a coin of 3 (reducing to $f(3) = 1$) plus one more coin — two coins of denomination 3 total. This table, built mechanically from the five-step method, requires no special insight beyond correctly executing the method itself.

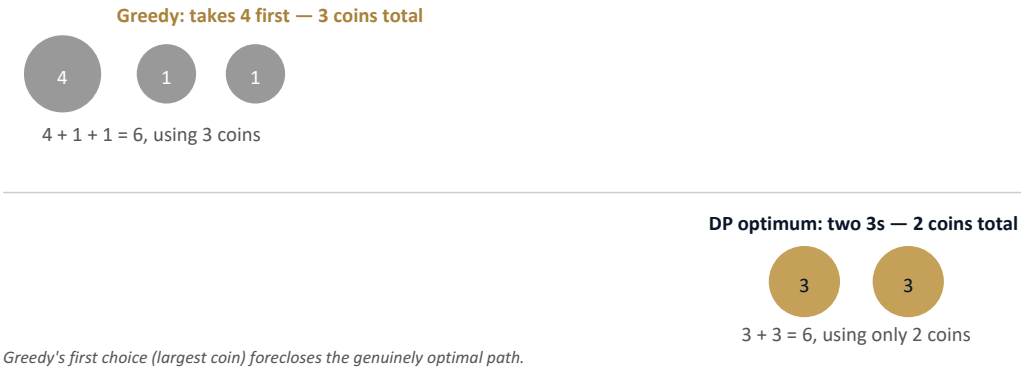
Why greedy fails on coin change: a concrete counterexample

A tempting shortcut, applicable to some coin systems, always takes the largest available coin that still fits, repeating until the amount is reached. For denominations $[1, 3, 4]$ and target amount 6, this greedy strategy takes a coin of 4 first (the largest that fits), leaving 2, which then requires two coins of denomination 1 — 3 coins total. But this lecture's DP table already proved the true optimum is 2 coins, using two coins of denomination 3. Greedy's answer here is not merely unproven to be optimal — it is demonstrably, concretely wrong, a genuine counterexample this course can point to directly.

Why this matters: previewing Week 10's central distinction

This is the central distinction Week 10 formalizes. Dynamic programming considers every valid option at each decision point and retains whichever leads to the true global optimum, a correctness guarantee proven directly by the optimal-substructure argument established in Lecture 2. Greedy algorithms make a single, irrevocable, locally-best choice at each step and never reconsider it, which is only correct when a considerably stronger structural property holds — Week 10 defines this property, the greedy-choice property, precisely. Coin change with denominations $[1, 3, 4]$ lacks this property, as this lecture's counterexample demonstrated concretely; coin change with standard currency denominations like $[1, 5, 10, 25]$ happens to possess it, meaning the identical greedy algorithm is correct on one input family and provably wrong on another — underscoring why this property must be verified, never assumed.

The picture: greedy's path vs. DP's true optimum



Greedy's first choice (largest coin) forecloses the genuinely optimal path.

Seeing both paths laid out side by side makes the failure visually immediate: greedy's locally-reasonable first choice (the largest coin, 4) actively forecloses the genuinely optimal path, which required choosing a smaller coin (3) at the very first step — exactly the kind of short-sightedness dynamic programming's exhaustive-but-organized approach is structurally immune to.

Common mistakes when applying the method, worth watching for

Three mistakes recur often enough to name explicitly. Skipping step 1 — writing down a recurrence before precisely defining, in words, what f of its parameters actually represents — is the single most common source of a subtly wrong recurrence, since an imprecise definition makes it easy to overlook a case. An incorrect fill order computes some $f(a)$ before a value it depends on, $f(a-c)$, has actually been computed, silently using a stale or default value rather than crashing outright, which can produce a plausible-looking but wrong answer that is hard to detect without careful testing. Forgetting that reconstruction is a genuinely separate step — a table storing only optimal values cannot recover the actual chosen solution afterward without the specific extra bookkeeping Lecture 2 and Lecture 3 both demonstrated — is a common oversight when a problem statement asks for the solution itself, not merely its value.

Consolidating Week 9's four problems into one table

Consolidating every dynamic programming problem covered this week into one table reveals the shared underlying shape beneath surface-level differences: each problem is defined by one or two subproblem parameters, each has a recurrence relating it to strictly smaller instances of itself, and each achieves polynomial cost by ensuring every distinct subproblem is computed exactly once. This table is worth returning to whenever a genuinely new problem is encountered — checking whether it fits this same shape is precisely what the five-step method systematizes.

MTech addition — DP on trees, a preview

```
def max_independent_set(node):
    if node is None:
        return 0
    if node.left is None and node.right is None:
        return node.value            # a leaf: including it is always at least as good
    include = node.value
    for child in (node.left, node.right):
        if child:
            include += sum(max_independent_set(gc) for gc in (child.left, child.right) if gc)
    exclude = sum(max_independent_set(child) for child in (node.left, node.right) if child)
    return max(include, exclude)
```

Every dynamic programming problem covered this week indexed its subproblems by an integer or a pair of integers. This preview, maximizing the total value of a subset of tree nodes with no two adjacent nodes both selected, indexes its subproblems by tree nodes from Week 7 instead. The identical five-step method still applies directly: the subproblem is "the best achievable value within this node's subtree," the recurrence considers including or excluding the current node, and the base case is a leaf node. This is worth internalizing as this lecture's final, broadening point: dynamic programming is not restricted to arrays and grids — it applies to any structure where subproblems relate to strictly smaller instances, including trees here and, as Week 12 will show, graphs as well.

Reconstructing coin change's actual coins

```
def coin_change_reconstruct(coins, target):
    f = [0] * (target + 1)
    choice = [None] * (target + 1)
    for a in range(1, target + 1):
        best = float('inf')
        for c in coins:
            if c <= a and 1 + f[a - c] < best:
                best = 1 + f[a - c]
                choice[a] = c
        f[a] = best
    result, a = [], target
    while a > 0:
        result.append(choice[a])
        a -= choice[a]
    return f[target], result
```

This reconstruction, applying step 5 of the general method to coin change specifically, tracks which coin achieved the minimum at each amount alongside the minimum value itself. Walking backward from the target, repeatedly subtracting the recorded coin, recovers the actual sequence of coins used — for target 6 with denominations $[1, 3, 4]$, this correctly returns `[3, 3]`, the two coins of denomination 3 already identified as optimal earlier in this lecture.

Exercise

As an exercise, apply the five-step method in writing, before writing any code, to a new problem: given a list of numbers, find the length of the longest strictly increasing subsequence. Define the subproblem precisely, derive the recurrence relating it to smaller instances, and identify the base case, exactly following this lecture's structure. Separately, implement coin change's reconstruction step and confirm which specific coins are used to make amount 6 from denominations $[1, 3, 4]$, verifying it matches the two coins of denomination 3 this lecture identified as optimal.

Where this method will reappear later in the course

This five-step method is not confined to Week 9. Week 12's graph algorithms, particularly shortest-path problems, reuse this exact method, with subproblems indexed by graph nodes rather than integers, pairs, or tree nodes. Week 14's material on parallel and randomized algorithms touches on parallelizing certain dynamic programming fill orders by recognizing that subproblems at the same "level" of dependency are independent of each other and can be computed simultaneously. This method is best understood as a recurring analytical lens, applied throughout the remainder of this course whenever a new problem exhibits repeated, smaller versions of itself — not a technique whose relevance ends when this week does.

Quiz — apply the method, cold

Using this lecture's coin change recurrence and the table built up through $f(6)$, compute $f(7)$ before checking the answer.

Quiz — answer

Applying the recurrence: $f(7) = 1 + \min(f(6), f(4), f(3)) = 1 + \min(2, 1, 1) = 2$. This minimum is achieved by either using a coin of denomination 4 (reducing to the already-computed $f(3) = 1$) or a coin of denomination 3 (reducing to $f(4) = 1$) — both paths correctly arrive at 2 total coins, confirming the table built earlier in this lecture extends consistently to a new target amount.

A closing check: the five-step method as a review checklist

As a final, practical closing check: before submitting any dynamic programming solution, in this course's assignments or in any future context, verify that the subproblem definition, the recurrence, and the base case can each be stated cleanly in one sentence. If any of the three resists a clean one-sentence statement, the solution likely contains an unrecognized edge case or a recurrence that was never properly verified against a concrete example, the way this lecture verified coin change's table by hand before trusting it. This checklist costs only a few minutes to run through and catches a large fraction of dynamic programming bugs before the code is ever executed at all.

Summary

The five-step method — precisely define the subproblem, find the recurrence, identify the base case, decide the fill order or add a cache, and reconstruct the actual solution if needed — applies systematically to any new dynamic programming problem, not merely the four examples covered this week. Coin change, worked through live using this method, demonstrates it end to end on a genuinely new problem, and the concrete counterexample showing greedy's failure on denominations $[1, 3, 4]$ previews Week 10's central distinction directly: dynamic programming's exhaustive-but-organized correctness guarantee, versus greedy's speed, purchased only when a specific, provable structural property actually holds. Every problem covered this week shares one underlying shape — subproblems, a recurrence connecting them, and a total cost dominated by the number of distinct subproblems times the work needed per subproblem. This closes Week 9. Next week introduces greedy algorithms, faster in general but correct only under conditions that must be proven, never merely assumed.